

Desarrollo Web Entorno Servidor
Unidad 8: Caso 1

**CREACIÓN DE UNA APLICACIÓN DE GESTIÓN DE
EVENTOS CON ACTUALIZACIÓN DINÁMICA EN
CLIENTE Y SERVIDOR**

Índice

1. [Introducción](#)
2. [Descripción general de la aplicación](#)
 - 2.1 [Finalidad de la aplicación](#)
 - 2.2 [Funcionalidades principales](#)
3. [Responsabilidades del servidor y del cliente](#)
 - 3.1. [Funciones del servidor con Node.js](#)
 - 3.2. [Funciones del cliente con Vue.js](#)
4. [Generación de páginas dinámicas con EJS](#)
5. [Integración entre cliente y servidor mediante API REST y Fetch](#)
6. [Ventajas de combinar tecnologías de servidor y cliente](#)
7. [Conclusión](#)
8. [Fuentes consultadas](#)

1. Introducción

En esta tarea se plantea el diseño de una aplicación web orientada a la gestión de eventos culturales, con el objetivo de combinar tecnologías de ejecución en servidor y en cliente dentro de un mismo proyecto. La propuesta parte de la necesidad de crear una solución dinámica e interactiva que permita consultar, crear y modificar eventos, así como gestionar la reserva de entradas de forma ágil y actualizada.

Para ello, se plantea el uso de Node.js en el lado del servidor, encargado de la lógica de negocio, el acceso a los datos y la validación de la información recibida, junto con EJS para la generación inicial de páginas dinámicas. En el lado del cliente, se utilizará Vue.js para mejorar la interacción con el usuario y permitir la actualización dinámica del contenido sin necesidad de recargar completamente la página.

De este modo, la aplicación no solo facilita la organización de actividades culturales como conciertos, exposiciones, obras de teatro o talleres, sino que también sirve como ejemplo práctico de cómo integrar tecnologías de servidor y cliente para desarrollar una experiencia web más moderna, fluida y eficiente.

2. Descripción general de la aplicación

La aplicación propuesta se plantea como una plataforma web destinada a la gestión de eventos culturales, combinando funciones de consulta para los usuarios y herramientas de administración para los responsables de la organización. Su diseño busca ofrecer una experiencia clara, dinámica y actualizada, permitiendo tanto la publicación de actividades culturales como la gestión de reservas en aquellos casos en los que el aforo sea limitado.

2.1. Finalidad de la aplicación

La finalidad principal de esta aplicación es centralizar la publicación, consulta y actualización de eventos culturales organizados por un centro cultural o una entidad local. A través de esta plataforma, los usuarios podrán acceder de forma sencilla a la información de actividades como conciertos, exposiciones, obras de teatro, talleres o proyecciones, consultando sus datos más relevantes y, cuando corresponda, realizando la reserva de entradas.

Además, la aplicación también está pensada como una herramienta de gestión interna para los administradores, ya que permitirá crear nuevos eventos, modificar la información de los existentes y mantener actualizada la programación cultural. De este modo, se cubren tanto las necesidades de difusión de la oferta cultural como las de organización y control de la información publicada.

2.2. Funcionalidades principales

Entre las funcionalidades principales de la aplicación se encuentra la visualización de eventos culturales, mostrando información como el título, la descripción, la fecha, la hora, la ubicación y la categoría de cada actividad. Esta presentación permitirá a los usuarios consultar fácilmente la programación disponible de forma ordenada y comprensible.

Junto a ello, la aplicación incluirá funciones de creación y modificación de eventos desde la parte administrativa, con el fin de mantener la información actualizada en todo momento. También se contempla la posibilidad de realizar reservas de entradas para aquellos eventos con plazas limitadas, mostrando de forma dinámica la disponibilidad restante. Como apoyo a la navegación, podrán incorporarse filtros por fecha o categoría, mejorando así la experiencia de uso y facilitando la localización de actividades concretas.

3. Responsabilidades del servidor y del cliente

En una aplicación web como la planteada, es necesario diferenciar con claridad las funciones que corresponden al servidor y las que recaen en el cliente. Aunque ambas partes trabajan de forma conjunta, cada una cumple un papel distinto dentro del sistema. El servidor se encarga de procesar la información, aplicar la lógica del negocio y garantizar que los datos se gestionen de forma correcta, mientras que el cliente se ocupa de la interacción directa con el usuario y de la actualización visual de la interfaz.

Esta separación de responsabilidades permite desarrollar una aplicación más organizada, eficiente y fácil de mantener, ya que cada tecnología se utiliza en el ámbito en el que ofrece mejores resultados. En este caso, Node.js asumirá las tareas propias del lado del servidor, mientras que Vue.js se utilizará en el navegador para mejorar la experiencia de uso y aportar dinamismo a la aplicación.

3.1. Funciones del servidor con Node.js

Node.js se utilizará en esta aplicación para gestionar todas las tareas relacionadas con el procesamiento interno de la información. Entre sus funciones principales se encuentra la recepción de solicitudes enviadas desde el navegador, la ejecución de la lógica de negocio y la respuesta adecuada según la operación solicitada. Por ejemplo, cuando un administrador cree un nuevo evento o modifique uno ya existente, será el servidor quien reciba esos datos, los procese y determine si cumplen las condiciones necesarias para ser almacenados.

Otra de sus responsabilidades fundamentales será la validación de datos. Antes de guardar o actualizar la información de un evento, el servidor deberá comprobar que

campos como el título, la fecha, la ubicación o el número de plazas disponibles contienen valores correctos y coherentes. Esta validación es importante para evitar errores, datos incompletos o información inconsistente dentro del sistema.

Además, el servidor será el encargado de interactuar con la base de datos o sistema de almacenamiento utilizado por la aplicación. Desde él se realizarán las consultas necesarias para recuperar la lista de eventos, guardar nuevos registros, modificar los existentes o actualizar la disponibilidad de entradas cuando se realice una reserva. De este modo, Node.js actúa como el núcleo lógico de la aplicación, centralizando el control de la información y garantizando el correcto funcionamiento del sistema.

3.2. Funciones del cliente con Vue.js

En el lado del cliente, Vue.js se empleará para gestionar la parte visible e interactiva de la aplicación. Su función principal será facilitar la comunicación entre el usuario y la interfaz, permitiendo que determinadas acciones se reflejen de forma inmediata en pantalla sin necesidad de recargar por completo la página. Esto mejora la experiencia de navegación y hace que la aplicación resulte más fluida y cómoda de utilizar.

Gracias a Vue.js, será posible controlar formularios de creación o edición de eventos, aplicar filtros de búsqueda y actualizar automáticamente la información mostrada cuando se produzca algún cambio, como por ejemplo una nueva reserva o la modificación de un evento. En lugar de volver a cargar toda la página, solo se actualizarán los componentes necesarios, lo que hace que la interacción sea más rápida y eficiente.

Además, Vue.js contribuye a una mejor organización del código del lado cliente, ya que permite estructurar la interfaz en componentes reutilizables y mantener sincronizados los datos con su representación visual. En esta aplicación, esto resultará especialmente útil para mostrar listados de eventos, mensajes de confirmación, estados de disponibilidad o resultados filtrados de manera dinámica y clara para el usuario.

Aspecto	Servidor (Node.js)	Cliente (Vue.js)
Gestión de datos	Procesa y almacena la información de los eventos	Muestra los datos al usuario
Validación	Comprueba que los datos recibidos sean correctos	Puede realizar validaciones básicas en formularios
Interacción con el usuario	Responde a las solicitudes enviadas desde el navegador	Gestiona clics, formularios y filtros
Actualización de contenido	Devuelve datos actualizados tras cada operación	Actualiza la interfaz sin recargar la página
Acceso a datos	Consulta y modifica la base de datos	No accede directamente a la base de

		datos
Presentación	Genera la página inicial con EJS	Modifica partes concretas de la interfaz dinámicamente

Tabla 1. Distribución de responsabilidades entre servidor y cliente

4. Generación de páginas dinámicas con EJS

Para la construcción inicial de la interfaz, la aplicación utilizará EJS como motor de plantillas en el lado del servidor. Esta tecnología permitirá generar páginas HTML dinámicas a partir de los datos almacenados en el sistema, de manera que el contenido mostrado al usuario no sea fijo, sino que se adapte a la información disponible en cada momento. Gracias a ello, la página principal de eventos podrá renderizarse desde el servidor con una estructura ya organizada antes de llegar al navegador.

Mediante EJS será posible recorrer la colección de eventos y mostrar cada uno de ellos dentro de la página usando bucles, lo que facilita la presentación de listados de forma automática y ordenada. Así, en lugar de escribir manualmente cada bloque de contenido, la plantilla podrá generar tantos elementos como eventos existan en el sistema. Esto resulta especialmente útil en una aplicación de este tipo, ya que la programación cultural puede variar con frecuencia y el número de eventos no será siempre el mismo.

Además, EJS permite utilizar estructuras condicionales para decidir qué información debe mostrarse en cada caso. Por ejemplo, se podrán presentar únicamente los eventos futuros, destacar aquellos que todavía dispongan de plazas libres o incluso ocultar determinadas opciones según el tipo de usuario que acceda a la aplicación. De esta forma, el contenido generado desde el servidor será más útil, claro y adaptado al contexto.

El uso de EJS en esta fase inicial también aporta una ventaja importante en la organización del proyecto, ya que separa la lógica de presentación del resto de la lógica del servidor. Esto favorece un desarrollo más limpio y mantenible, permitiendo construir una base sólida sobre la que después el cliente, mediante Vue.js, podrá aplicar actualizaciones dinámicas y mejorar todavía más la interacción del usuario.

5. Integración entre cliente y servidor mediante API REST y Fetch

Para que la aplicación funcione de forma dinámica y mantenga la información actualizada sin necesidad de recargar continuamente la página, será necesario establecer una comunicación fluida entre el cliente y el servidor. Esta integración se llevará a cabo mediante una API REST desarrollada en Node.js, que permitirá

intercambiar datos entre ambas partes de forma estructurada, y mediante el uso de Fetch en el navegador para realizar solicitudes asíncronas.

La API REST actuará como un punto de conexión entre la interfaz y la lógica del sistema. A través de distintas rutas, el cliente podrá solicitar información sobre los eventos disponibles, enviar nuevos datos cuando se cree un evento, modificar los ya existentes o actualizar la disponibilidad de entradas tras una reserva. De este modo, cada operación importante de la aplicación quedará asociada a una petición concreta al servidor, siguiendo una organización clara y coherente.

Por ejemplo, una solicitud de tipo GET podrá utilizarse para obtener la lista de eventos almacenados, mientras que una petición POST servirá para crear un nuevo evento. Del mismo modo, una petición PUT permitirá modificar la información de un evento ya existente, como su fecha, ubicación o número de plazas disponibles. Gracias a esta estructura, la comunicación entre cliente y servidor será más ordenada y facilitará el mantenimiento de la aplicación.

Desde el lado del cliente, Fetch permitirá enviar estas solicitudes de forma asíncrona, es decir, sin bloquear la navegación ni obligar a recargar la página completa. Esto resulta especialmente útil en acciones como aplicar filtros, registrar una reserva o actualizar la información de un evento, ya que los cambios podrán reflejarse de forma inmediata en la interfaz. Así, cuando un usuario reserve una entrada, el navegador podrá enviar la solicitud al servidor, recibir la respuesta con la disponibilidad actualizada y modificar únicamente la parte de la página donde se muestra esa información.

La combinación de API REST y Fetch hace posible una experiencia de uso mucho más ágil, ya que reduce recargas innecesarias y permite que la información esté sincronizada en todo momento entre servidor y cliente. Además, esta forma de trabajo favorece una mejor separación de responsabilidades, ya que el servidor se centra en procesar y validar los datos, mientras que el cliente se encarga de mostrarlos y actualizarlos visualmente de manera inmediata.

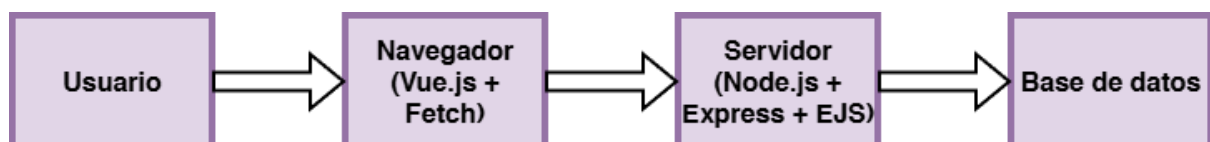


Figura 1. Esquema general de funcionamiento de la aplicación

En este esquema se representa el flujo básico de interacción entre el usuario, la interfaz ejecutada en el navegador, el servidor que procesa la lógica de la aplicación y el sistema de almacenamiento de datos.

6. Ventajas de combinar tecnologías de servidor y cliente

La combinación de tecnologías de servidor y cliente aporta una serie de ventajas importantes en el desarrollo de aplicaciones web modernas. En primer lugar, permite repartir las responsabilidades de forma más adecuada, haciendo que cada parte del sistema se encargue de las tareas para las que está mejor preparada. Mientras el servidor asume la lógica de negocio, la validación de datos y el acceso al almacenamiento, el cliente se ocupa de la interacción con el usuario y de la actualización visual de la interfaz.

Esta distribución mejora tanto la organización interna del proyecto como su mantenimiento a medio y largo plazo. Al separar claramente las funciones de cada tecnología, el código resulta más limpio, más comprensible y más fácil de ampliar en el futuro. Además, se reduce la dependencia de recargas completas de página, lo que permite ofrecer una navegación más rápida y fluida.

Otra ventaja importante es la mejora de la experiencia de usuario. Gracias al trabajo conjunto entre Node.js, EJS y Vue.js, la aplicación puede ofrecer una primera carga estructurada desde el servidor y, a partir de ahí, actualizar la información de forma dinámica en el navegador. Esto hace que acciones como consultar eventos, aplicar filtros o reservar entradas se realicen de forma más cómoda, inmediata y visualmente clara.

Por último, la integración entre servidor y cliente favorece el desarrollo de aplicaciones más interactivas y adaptadas a las necesidades actuales. En lugar de limitarse a mostrar contenido estático, la aplicación puede responder a las acciones del usuario en tiempo real, mantener la información actualizada y ofrecer una experiencia más cercana a la de una plataforma web moderna. Por ello, la unión de estas tecnologías no solo mejora el funcionamiento técnico del sistema, sino también su utilidad práctica y su calidad final.

7. Conclusión

El desarrollo de una aplicación web de gestión de eventos culturales permite comprobar de forma práctica la utilidad de combinar tecnologías de servidor y de cliente dentro de un mismo proyecto. A lo largo de esta propuesta se observa que Node.js resulta adecuado para asumir la lógica de negocio, la validación de datos y la comunicación con el sistema de almacenamiento, mientras que Vue.js aporta una capa de interacción más dinámica y fluida en el navegador. EJS, por su parte, facilita la generación inicial de páginas dinámicas desde el servidor mediante plantillas con JavaScript embebido, y el uso de Fetch permite mantener la comunicación asíncrona entre cliente y servidor para actualizar la información sin recargas completas.

En conjunto, esta arquitectura permite desarrollar una aplicación más organizada, interactiva y adaptada a las necesidades actuales de la web. La separación de responsabilidades entre servidor y cliente no solo mejora el mantenimiento del proyecto, sino que también ofrece una experiencia de uso más clara, rápida y eficiente. Por ello, la integración de Node.js, EJS, Vue.js y Fetch constituye una solución adecuada para una plataforma de gestión de eventos culturales con reserva de entradas y actualización dinámica de la información.

8. Fuentes consultadas

- **Programming with Mosh.** *Node.js Tutorial for beginners: Learn Node in 1hr.* Youtube
https://youtu.be/TIB_eWDSMt4?si=gJSyN_jN3J1tjYh
- **freeCodeCamp.org.** *Learn Vue.js - Tutorial for beginners.* Youtube
<https://youtu.be/Kt2E8nblvXU?si=R1DRzC-OVt6k0T7I>
- **byDegraphe.** *Node.js Crash Course.* Youtube
<https://youtube.com/playlist?list=PLLfhpjqtVZ-EBHK9jSonlyOQbk8KI1g7Q&si=AdE8jCI-yp86OrTs>
- **EJS.** *Embedded JavaScript templates.* Documentación oficial de EJS.
<https://ejs.co/>
- **MDN Web Docs.** *Using the Fetch API.* Mozilla Developer Network.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- **Anson.** *Express JS Full Tutorial Playlist 2024.* Youtube
https://www.youtube.com/watch?v=P6RZfl8KDYc&list=PL_cUvD4qzbkwjmjy-KjbieZ8J9cGwxZpC
- **Express.** *Routing.* Documentación oficial de Express.
<https://expressjs.com/es/guide/routing.html>
- *Material de la unidad didáctica*